

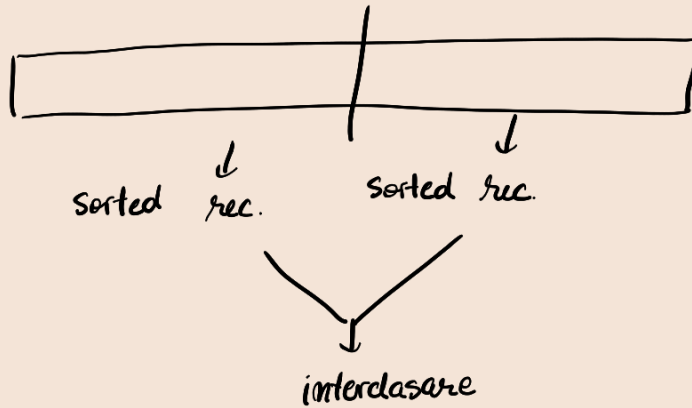
- Insertion
- Binary insertion.
- Quick: pivot $\begin{cases} \text{correctly} \\ \text{incorrectly} \end{cases}$ $\begin{cases} \text{max} \\ \text{switch the quick sort.} \end{cases}$

MERGE SORT

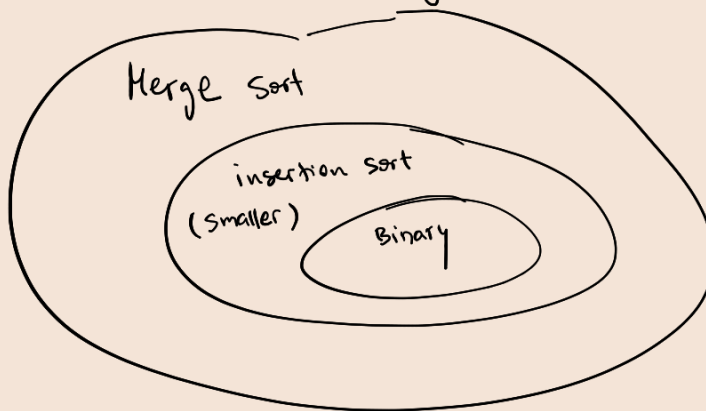
$\rightarrow m \cdot \log m$

$\rightarrow$ divide and conquer $\rightarrow m \cdot \log_2 m$ (Bc, Ac, Wc)

recursive
slice the list
additional memory } cons.



1) $\rightarrow$ merge-insertion sort. (faster than merge sort?)



Heap Sort (entire course on graphs)

2) Intro Sort
 $\hookrightarrow$ Wc of quicksort

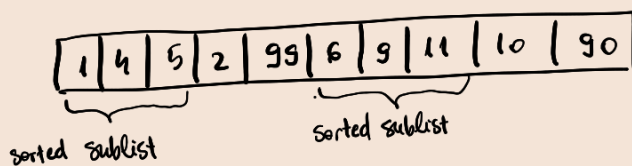
Tim Sort

- $\hookrightarrow$ merge sort, binary insertion sort
- $\hookrightarrow$ tries to use cache as much as possible.

Bc: $O(m)$
Ac: $O(m \log_2 m)$
Wc: $O(m \log_2 m)$

matrix $\rightarrow$ row faster (cache)

$\rightarrow$ column (no cache $\rightarrow$ ram $\rightarrow$ cache)



! Always merge lists with the same lengths
↳ minimise the merges.

If the sublists are sorted → do I merge them ?
↳ do I append them

memcpy → a region of mem, dest, size

Sorting algorithm comparison:

Python interpreter ⇒ slower, all of the libraries are written in C.

↳ sorted() → calls C

pages, not bytes make the big memory

Next: problem solving

< Backtracking (recap)
Dynamic programming